

FULL PROJECT DOCUMENTATION

Implement Client Script & UI Policy (Incident)

ServiceNow Platform Documentation

Team ID	SWTID-2026-1951
Platform	ServiceNow Personal Developer Instance (PDI)
Version	1.0
Team Members	Mohammed Akif A; Madhavansundar R; Karthikeyan S; Hayagreevan K; Amirthaa Nikash B

1. INTRODUCTION

Project Title: Implement Client Script & UI Policy (Incident)

Project Type: ServiceNow Incident form configuration using UI Policies and Client Scripts.

Team Members: Mohammed Akif A, Madhavansundar R, Karthikeyan S, Hayagreevan K, Amirthaa Nikash B.

2. PROJECT OVERVIEW

Purpose

The purpose is to improve Incident data entry by dynamically controlling fields and validating user input at the appropriate event. The project aims to reduce incorrect data, improve user guidance and keep the Incident form simple to use.

Features

- UI Policy to control field visibility, mandatory state and read-only state.
- UI Policy Action for the Urgency field.
- onChange Client Script for immediate guidance.
- onSubmit Client Script for save validation.
- onCellEdit Client Script for inline list validation.
- ServiceNow-native cloud deployment through a Personal Developer Instance.

3. ARCHITECTURE

Frontend / User Interface

The user interface is the ServiceNow Next Experience UI or Classic UI in a browser. Users work with the Incident form and Incident list view. The project does not use React or another standalone frontend framework.

Application Logic

Client-side JavaScript uses ServiceNow APIs such as `g_form` and `g_user`. UI Policies provide declarative field behaviour, while Client Scripts handle `onChange`, `onSubmit` and `onCellEdit` events.

Database

Incident records are stored in the ServiceNow **incident** table within the platform-managed relational data layer. Fields include caller, category, urgency, impact, priority and state.

Architecture Diagram

Project Design Phase Solution Architecture

Date	15 February 2025
Team ID	SWTID-2026-1951
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Solution Architecture:

Solution architecture is a complex process – with many sub-processes – that bridges the gap between business problems and technology solutions. Its goals are to:

- Find the best tech solution to solve existing business problems.
- Describe the structure, characteristics, behavior, and other aspects of the software to project stakeholders.
- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Example - Solution Architecture Diagram:

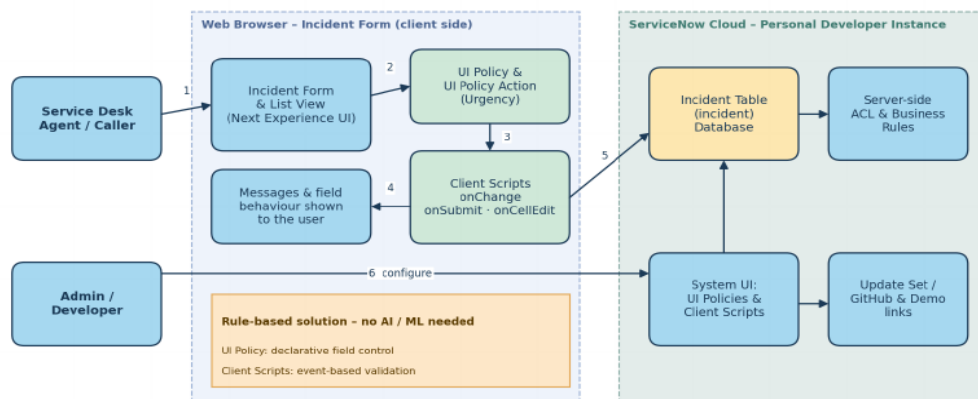


Figure 1: Architecture and data flow of the Client Script & UI Policy solution on the ServiceNow Incident form

1. The agent or caller opens a new or existing Incident in the browser.
2. The UI Policy engine sets field behaviour (mandatory, read-only, visible) as the form loads.
3. When a field such as Urgency changes, the UI Policy Action and onChange Client Script run instantly.
4. The user sees updated fields and on-screen messages without submitting the form.
5. On save, the onSubmit script validates the data; valid data is stored in the Incident table (list edits go through onCellEdit).
6. Admin / developer creates and maintains UI Policies and Client Scripts in System UI and shares the Update Set, demo and GitHub links.

Reference: <https://developer.servicenow.com/>

Figure 1. ServiceNow solution architecture supplied in the project documents.

4. SETUP INSTRUCTIONS

Prerequisites

- ServiceNow account with access to a Personal Developer Instance or equivalent development instance.
- Admin/developer permissions to create and maintain UI Policies and Client Scripts.

- Chrome, Edge or Firefox browser.
- Incident Management application/table enabled in the instance.

Installation / Configuration

- Open the ServiceNow Personal Developer Instance in a browser.
- Sign in using an authorized admin/developer account.
- Open the Incident application and verify access to the Incident form and list.
- Configure the required UI Policy on the Incident form.
- Configure the UI Policy Action for the Urgency field.
- Create the onChange Client Script for monitored field changes.
- Create the onSubmit Client Script for save validation.
- Create the onCellEdit Client Script for inline list validation.
- Test each rule with both valid and invalid scenarios.
- Use an Update Set when moving the configuration between instances.

Note: no local Node.js, MongoDB, npm or standalone server is required for this ServiceNow implementation.

5. FOLDER / CONFIGURATION STRUCTURE

Because this is a ServiceNow platform configuration rather than a MERN codebase, the equivalent structure is:

ServiceNow area	Project contents
Incident table / form	User-facing Incident fields and form behaviour.
System UI → UI Policies	Declarative rules controlling field state.
UI Policy Actions	Field-specific actions such as Urgency behaviour.
System Definition → Client Scripts	onChange, onSubmit and onCellEdit JavaScript rules.
Update Sets	Packaging and transferring configuration changes.
Attachments / repository	Supporting documentation and project files, where used.

6. RUNNING THE APPLICATION

There is no separate application server to start. The application runs inside the ServiceNow cloud instance.

Step	Action
1	Open the ServiceNow PDI URL in a supported browser.
2	Sign in with an authorized account.
3	Navigate to Incident → Create New to test the form.
4	Change fields covered by UI Policy/onChange logic and observe the response.
5	Attempt a valid and invalid save to test onSubmit validation.
6	Open the Incident list and test valid and invalid inline edits for onCellEdit.

7. API DOCUMENTATION

The project does not expose custom REST API endpoints. It uses ServiceNow client-side APIs within Client Scripts.

API / mechanism	Use in project	Purpose
g_form	GlideForm client API	Read/set form fields, display messages and control form behaviour.

API / mechanism	Use in project	Purpose
g_user	GlideUser client API	Access current-user information when required.
UI Policy engine	Platform mechanism	Apply mandatory/read-only/visible field state from conditions.
UI Policy Action	Platform mechanism	Apply field-level behaviour, including Urgency.
onChange	Client Script event	React when a monitored form field changes.
onSubmit	Client Script event	Validate the form before saving.
onCellEdit	Client Script event	Validate inline list edits.

8. AUTHENTICATION

Authentication is handled by the ServiceNow platform rather than custom application code. Authorized users sign in to the ServiceNow instance, and role-based access controls/ACLs govern access to configuration and Incident data. The project documents specify admin/developer access for creating scripts and policies. Credentials are not recorded in this document.

9. USER INTERFACE

The main interface is the ServiceNow Incident form and Incident list view. The user experience includes dynamic field states, immediate messages, Urgency behaviour, save validation and inline list validation.

UI feature	User-visible behaviour
Dynamic field state	Fields become mandatory, read-only or visible according to policy conditions.
Urgency control	Urgency field behaviour changes according to the configured UI Policy Action.
Immediate guidance	onChange Client Script responds without submitting the form.
Save validation	onSubmit blocks invalid data and shows a message.
Inline validation	onCellEdit rejects invalid list edits.

The uploaded project package did not include standalone screenshots of the final ServiceNow UI; the architecture figure and UAT test matrix are included as available project evidence.

10. TESTING

Testing covers both positive and negative scenarios for UI Policies and Client Scripts, plus response-time and rapid-change checks.

Test category	Coverage
Functional	FT-01 to FT-05: UI Policy, Urgency action, onChange, onSubmit and onCellEdit.
Performance	PT-01 to PT-03: response time, rapid field changes and multiple inline edits.
UAT	20 test cases across UI Policy, Urgency, onChange, onSubmit, onCellEdit and configuration testing.
UAT result	20 passed, 0 failed, 0 not tested in the supplied UAT execution report.

Defect status

The supplied UAT execution report records 21 resolved/closed defect items across severity levels: 2 By Design, 1 Duplicate, 1 External, 13 Fixed, 1 Not Reproduced and 3 Won't Fix. The report does not identify these as open test failures.

11. SCREENSHOTS OR DEMO

The supplied package contains the solution architecture diagram and testing documentation but no standalone final-UI screenshots or project-demo URL. A live demonstration can be performed in the ServiceNow PDI using the steps in Section 6.

Project Design Phase Solution Architecture

Date	15 February 2025
Team ID	SWTID-2026-1951
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Solution Architecture:

Solution architecture is a complex process – with many sub-processes – that bridges the gap between business problems and technology solutions. Its goals are to:

- Find the best tech solution to solve existing business problems.
- Describe the structure, characteristics, behavior, and other aspects of the software to project stakeholders.
- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Example - Solution Architecture Diagram:

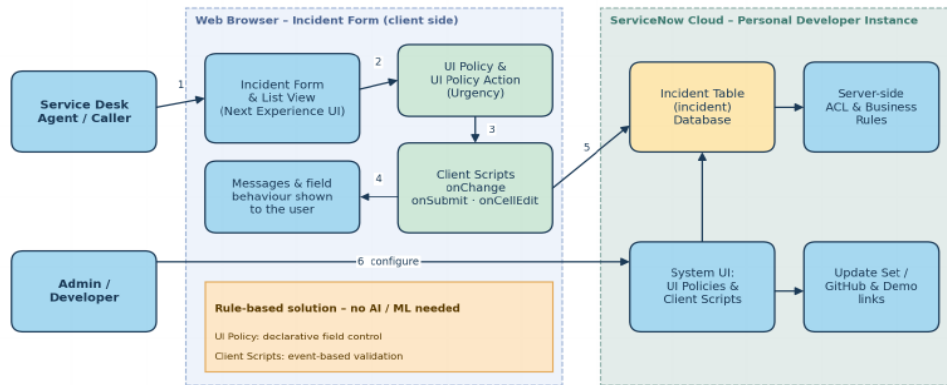


Figure 1: Architecture and data flow of the Client Script & UI Policy solution on the ServiceNow Incident form

1. The agent or caller opens a new or existing Incident in the browser.
2. The UI Policy engine sets field behaviour (mandatory, read-only, visible) as the form loads.
3. When a field such as Urgency changes, the UI Policy Action and onChange Client Script run instantly.
4. The user sees updated fields and on-screen messages without submitting the form.
5. On save, the onSubmit script validates the data; valid data is stored in the Incident table (list edits go through onCellEdit).
6. Admin / developer creates and maintains UI Policies and Client Scripts in System UI and shares the Update Set, demo and GitHub links.

Reference: <https://developer.servicenow.com/>

Available project evidence: solution architecture/data-flow diagram.

12. KNOWN ISSUES

- No open functional test failures are recorded in the supplied UAT execution summary.
- The solution depends on ServiceNow instance availability and appropriate roles/ACLs.
- Client-side rules provide user guidance and validation; stronger enforcement may require server-side controls for critical business rules.

PROJECT OVERVIEW SCREENSHOT

SkillWallet project overview for “Implement Client Script & UI Policy (Incident)”, showing the project description, team members, milestones, progress and system requirements.

SKILLWALLET

Project Overview

Dashboard

Skill Bank

Skill Wallet

Skill Card

Skill Credits

Dashboard > Skillwallet > Progri

0 AI Token

300 Credits

MA

Implement Client Script & UI Policy (Incident)

Team

Overview

Workspace

Ki

Add Demo and GitHub links!

Please update the demo and GitHub links so that your mentor can review and evaluate your project.

Information Technology

Group

This project focuses on implementing **Client Scripts and UI Policies** on the ServiceNow **Incident** form to improve user experience, enforce field behavior, and reduce incorrect data entry. The implementation dynamically controls form fields based on user actions and incident conditions without requiring the form to be submitted.

Skills Required

ServiceNow

Mentor

No mentor assigned yet

Team Members

M Mohammed Akif A TeamLead

M Madhavansundar R Member

K Karthikeyan S Member

H Hayagreevan K Member

A Amirthaa Nikash B Member

Project Stats

Epics

Total Epics: 7

Stories & Subtasks

Total Tasks: 11

Total Subtasks: 0

System Requirements

Instructions

- **ServiceNow Instance** – Personal Developer Instance (PDI) or any ServiceNow Instance with Incident Management enabled.
- **User Access** – Admin or developer role with permission to create and configure Client Scripts and UI Policies.
- **Incident Module** – Access to the Incident application and Incident form.
- **Browser** – Modern web browser such as Google Chrome, Microsoft Edge, or Firefox.
- **Scripting Support** – Basic knowledge of JavaScript and ServiceNow APIs.

Project Progress

50%

Add Demo Link

Add GitHub

Milestone 1: Create UI Policy on Incident

Complexity: Easy

Duration: 20m

Progress: 50%

Stories: 1

Milestone 2: Create UI Policy Action – Urgency

Milestone 3: Create onChange Client Script

Milestone 4: Create onSubmit Client Script (Save Validation)

Milestone 5: Create onCellEdit Client Script

Milestone 6: Testing the Configuration

Conclusion

Figure: Supplied SkillWallet project overview screenshot.

- The uploaded documentation does not contain the actual Client Script source listings, so this document describes intended behaviour rather than reproducing code that was not supplied.

13. FUTURE ENHANCEMENTS

- Extend the configuration to additional ServiceNow tables such as Problem and Change.
- Add more validation rules based on real service-desk error patterns.
- Create automated regression tests for repeated Incident scenarios.
- Use ServiceNow reports/dashboards to monitor data-quality and incident-processing metrics.
- Package and standardize the configuration as a reusable Update Set.
- Add server-side validation/business rules for critical data constraints.

REFERENCES

- <https://developer.servicenow.com/>
- <https://docs.servicenow.com/>
- <https://c4model.com/>
- <https://www.atlassian.com/agile/project-management>
- <https://www.atlassian.com/agile/tutorials/burndown-charts>